



UMCS

UNIwersytet Marii Curie-Skłodowskiej
w Lublinie

Wydział Matematyki, Fizyki i Informatyki

Kierunek: **Geoinformatyka**

Specjalność:

Michał Jan Wilkos

Nr albumu: 307370

Projekt i implementacja oprogramowania do analizy przestrzenno-czasowej danych treningowych z urządzeń GPS

Design and implementation of software for spatial-temporal
analysis of training data from GPS devices

Praca magisterska
napisana w Katedrze Oprogramowania Systemów Informatycznych
Instytutu Informatyki
pod kierunkiem dr Michała Klisowskiego

Lublin 2026

Spis treści

Wstęp	5
1 Teoria (do poprawy)	7
1.1 Pamięć (do poprawy)	7
1.1.1 Pamięć podręczna i hierarchia pamięci	7
1.1.2 Równoległe wykonywanie operacji	8
1.1.3 Wektoryzacja	9
1.2 GPS	11
1.2.1 Zasada działania	11
1.2.2 Formaty danych GPS	12
2 HPC i Python	17
2.1 HPC	17
2.1.1 Języki stosowane w HPC	19
2.1.2 Wbudowane struktury danych języka Python a wydajność obliczeniowa	19
2.1.3 NumPy	21
2.2 Pomocnicze biblioteki	23
3 Implementacja kodu	25
3.1 Struktura projektu	25
3.2 Implementacja kodu do parsowania plików TCX	26
3.3 Implementacja kodu przetwarzającego dane treningowe	29
3.3.1 Funkcje odpowiedzialne za obliczanie dystansu oraz przewyższeń	29
3.3.2 Analiza parametru tętna serca	30
3.3.3 Analiza wydolności biomechanicznej i tlenowej	33
Bibliografia	39
Spis listingów	41

Wstęp

W czasach rosnącej świadomości na temat zdrowego trybu życia oraz dynamicznego rozwoju technologii pomiaru sesji treningowych coraz więcej osób decyduje się na monitorowanie swoich aktywności fizycznych, a popularność zegarków sportowych i urządzeń z modułami GPS rośnie z roku na rok. Jednak generowanie ogromnych ilości danych o przebytych dystansach i parametrach treningowych niesie ze sobą problemy nie tylko w kontekście interpretacji, ale przede wszystkim obliczeń, szczególnie w przypadku przetwarzania informacji przestrzenno-czasowych.

W procesie tworzenia oprogramowania analitycznego ważną rolę odgrywa wybór odpowiedniej metody przetwarzania dużych zbiorów danych. Chociaż język Python oferuje mnogość narzędzi, wiele rozwiązań różni się pod względem efektywności czasowej i pamięciowej. Często stosowane standardowe struktury danych, takie jak listy, mogą okazać się niewystarczające przy skomplikowanych i obciążających pamięć obliczeniach. Te zagadnienia oraz chęć sprawdzenia różnic w wydajności stanowiły inspirację do stworzenia aplikacji, która posłuży jako środowisko testowe dla tych metod.

Praca będzie się koncentrować na opracowaniu oraz zaimplementowaniu oprogramowania do analizy przestrzenno-czasowej danych treningowych, z uwzględnieniem porównania efektywności różnych metod algorytmów analitycznych. Głównym założeniem jest nie tylko stworzenie funkcjonalnego narzędzia, ale także zbadanie i zestawienie wydajności operacji wykonywanych na podstawowych strukturach danych w porównaniu do rozwiązań wektorowych.

Struktura pracy zostanie podzielona na trzy rozdziały. Pierwszy z nich omówi wykorzystaną technologię oraz charakterystykę przetwarzania danych GPS, ze wskazaniem na różnice pomiędzy klasycznym modelem programowania a podejściem wektorowym. Drugi rozdział będzie przedstawiać założenia projektowe aplikacji oraz metodologię przeprowadzania testów porównawczych. W ostatnim rozdziale przedstawione zostaną aspekty implementacyjne, w tym organizacja kodu oraz napotkane problemy, a jego ważną częścią będzie stanowić prezentacja wyników analizy porównawczej wydajności zastosowanych rozwiązań w odniesieniu do czasu przetwarzania i wykorzystania zasobów systemowych.

Rozdział 1

Teoria (do poprawy)

Ten rozdział poruszy tematy dotyczące pamięci, podstaw przetwarzania danych, które stanowią bazę do zrozumienia mechanizmów optymalizacji oprogramowania, a także przybliży architektury systemów nawigacji satelitarnej oraz różne formaty przechowywane dane przestrzenne.

1.1 Pamięć (do poprawy)

1.1.1 Pamięć podręczna i hierarchia pamięci

Współczesne procesory wykonują obliczenia szybciej, niż system pamięci jest w stanie dostarczyć im dane. Jest to problem znany jako „wąskie gardło” występujący w architekturze von Neumanna, według której dane są przechowywane w pamięci wraz z instrukcjami kodu programu[1]. Problem ten określa się również jako ścianę pamięci (ang. *memory wall*). To sprawia, że w aplikacjach przetwarzających duże ilości danych problemem staje się opóźnienie w dostępie do pamięci RAM, a nie moc obliczeniowa rdzenia.

Aby złagodzić ten problem, architektury komputerowe wykorzystują wielopoziomową pamięć podręczną, która stanowi swoisty bufor pomiędzy procesorem a pamięcią RAM. Dzieli się ona na trzy poziomy, z czego pierwszy z nich jest najszybszy, a trzeci – najwolniejszy[1]. Jest to mniejsza pamięć, ale znacznie szybsza od pamięci operacyjnej komputera. W celu uzyskania wysokiej wydajności często zarządza się danymi tak, aby procesor jak najczęściej znajdował potrzebne informacje w pamięci podręcznej, ograniczając kosztowne pobrania z pamięci głównej RAM.

Fundamentem wydajnego wykorzystania pamięci podręcznej jest zasada lokalności odwołań:

- Lokalność czasowa: Dane, do których uzyskano dostęp, prawdopodobnie będą potrzebne ponownie w niedługim czasie.

- Lokalność przestrzenna: Jeśli program odwołuje się do określonego adresu pamięci, z dużym prawdopodobieństwem wkrótce odwoła się do sąsiednich adresów[1].

Z perspektywy programisty i wyboru struktur danych w projekcie, lokalność przestrzenna ma bardzo ważne znaczenie. Procesory nie pobierają z pamięci pojedynczych bajtów, lecz całe linie pamięci podręcznej. Struktury takie jak tablice, gdzie elementy są ułożone w pamięci w kolejności jeden za drugim, są pod tym względem bardzo wydajne. Pobranie pierwszego elementu automatycznie ładuje do cache'u kolejne, co pozwala na płynne dostarczanie danych[2].

Struktury oparte na węzłach i wskaźnikach, takie jak listy połączone lub drzewa charakteryzują się rozrzuceniem elementów po sterzie pamięci. Przejście do kolejnego elementu wymaga skoku do losowego adresu, co często skutkuje chybieniem pamięci podręcznej i wstrzymaniem pracy procesora na czas pobrania danych z RAM[2].

Współczesne procesory wielordzeniowe zazwyczaj implementują trójpoziomowy system pamięci podręcznej. Pamięć podręczna pierwszego poziomu jest najszybszą i najmniejszą spośród wszystkich trzech poziomów. Dzieli się na dwie niezależne sekcje – L1i dla instrukcji oraz L1d dla danych. Taki podział zapobiega konfliktom między pobieraniem kodu a odczytem danych operacyjnych. Rozmiar tej pamięci waha się od 32 do 48 KB na rdzeń. [1]

Pamięć drugiego poziomu stanowi bufor między pamięcią L1 a wolniejszymi zasobami. Jej pojemność waha się od 256 KB do 2 MB na rdzeń. W przypadku „chybienia” w L1, procesor szuka danych w L2, co zapobiega wstrzymaniu pracy.[1]

Pamięć trzeciego poziomu jest największą pamięcią podręczną w procesorze. Zazwyczaj jest ona współdzielona przez wszystkie rdzenie w procesorze. Jej rozmiar może wynosić od kilkunastu do ponad stu megabajtów. Głównym zadaniem pamięci podręcznej trzeciego poziomu jest synchronizacja danych między rdzeniami, a także redukcja odwołań do pamięci RAM.[1]

Ważną kwestią do poruszenia w tym temacie jest mechanika transferu danych. System pamięci nie przesyła pojedynczych bajtów czy zmiennych. Komunikacja między pamięciami RAM, L3, L2 i L1 odbywa się za pomocą tzw. linii pamięci podręcznej (ang. *cache lines*). Oznacza to, że pojedyncze odwołanie do pamięci przez procesor powoduje pobranie całego 64-bajtowego bloku.[1]

1.1.2 Równoległe wykonywanie operacji

Przez dekady wzrost wydajności aplikacji opierał się na zwiększaniu częstotliwości taktowania procesorów, lecz ograniczenia architektoniczne sprawiły, że dalsze zwiększanie prędkości pojedynczego rdzenia stało się nieefektywne. Zamiast tego, zaczęto zwiększać liczbę rdzeni w procesorach, co doprowadziło do zmiany w projektowaniu systemów i aplikacji – przyspieszania pojedynczych instrukcji w stronę równoległych obliczeń[3].

Do klasyfikacji architektur równoległych stosuje się taksonomię zaproponowaną przez Michaela Flynna w 1966 roku

- SISD (ang. *Single Instruction, Single Data*): Architektura von Neumanna, gdzie jeden procesor wykonuje jedną instrukcję na jednej danej. Jest to model sekwencyjny.
- SIMD (ang. *Single Instruction, Multiple Data*): Jedna instrukcja sterująca wykonuje tę samą operację na wielu elementach danych jednocześnie. Jest to podstawa wektoryzacji.
- MISD (ang. *Multiple Instruction, Single Data*): Wiele instrukcji operuje na jednej danej.
- MIMD (ang. *Multiple Instruction, Multiple Data*): Wiele procesorów lub rdzeni wykonuje niezależnie różne instrukcje na różnych danych. Jest to podstawowy model równoległości[3].

Architektura MIMD jest realizowana w obecnych komputerach za pomocą procesorów wielordzeniowych. W kontekście analizy danych treningowych pozwala to na podział pracy na mniejsze, niezależne zadania. System może analizować kilka niezależnych plików jednocześnie, gdzie każdy plik obsługiwany jest przez osobny wątek procesora. W związku z tym system może działać w przykładowy sposób:

- Rdzeń 1 analizuje trasę biegu z poniedziałku.
- Rdzeń 2 analizuje trasę rowerową z wtorku.
- Rdzeń 3 analizuje spacer ze środy.

Ponieważ wynik analizy jednego działania nie zależy od wyniku drugiego, rdzenie te nie muszą się ze sobą komunikować ani na siebie czekać.[3]

1.1.3 Wektoryzacja

Według taksonomii zaproponowanej przez Michaela J. Flynna, tradycyjne jednostki obliczeniowe operują w modelu SISD. Oznacza to, że pojedyncza instrukcja maszynowa modyfikuje stan pojedynczego parametru w jednym cyklu maszynowym. Alternatywą, która zdominowała współczesne jednostki CPU, jest model SIMD. Paradygmat ten eksploatuje zjawisko równoległości na poziomie danych (ang. *Data-Level Parallelism*), umożliwiając zaaplikowanie tej samej operacji (np. dodawania, mnożenia) do wektora elementów rozlokowanych w dedykowanym, szerokim rejestrze sprzętowym.[3]

Zysk wydajnościowy płynący z wektoryzacji można opisać za pomocą prawa Amdahla. Jeżeli P oznacza ułamek programu podatny na zrównoleglenie wektorowe, a N

oznacza liczbę elementów przetwarzanych jednocześnie w jednym rejestrze wektorowym, to teoretyczne maksymalne przyspieszenie S (ang. *speedup*) określa wzór:

$$S = \frac{1}{(1 - P) + \frac{P}{N}}$$

Z powyższego równania wynika, że skuteczność technologii SIMD jest ściśle zdeterminowana przez to, jak duża część algorytmu daje się wyrazić w postaci niezależnych operacji wektorowych.[3]

Sama translacja instrukcji na postać wektorową nie gwarantuje proporcjonalnego przyspieszenia. Optymalizacja SIMD napotyka na dwa fundamentalne wyzwania, które muszą zostać uwzględnione w projektowaniu algorytmów. Najczęstszym powodem takiej nieefektywności jest niewystarczająca przepustowość szyny pamięci pomiędzy RAM a pamięcią podręczną procesora. Procesor jest w stanie wykonywać operacje SIMD znacznie szybciej, niż system pamięci dostarcza nowe dane.[3]

Bezpośrednią konsekwencją specyfiki ładowania danych wektorowych jest konieczność zmiany systemów projektowania struktur danych w pamięci (ang. *Data-Oriented Design*). Klasyczne programowanie obiektowe naturalnie układa dane w sposób AoS, czyli tablice struktur (ang. *Array of Structures*). Przylegające do siebie w pamięci bajty reprezentują różne atrybuty pojedynczego obiektu (np. x, y, z, kolor, czas). Utrudnia to załadowanie np. wyłącznie współrzędnych x z kolejnych elementów.

$$\text{AoS} = \left[\underbrace{(\text{Poz}_1, \text{Prę}_1)}_{\text{Obiekt}_1}, \underbrace{(\text{Poz}_2, \text{Prę}_2)}_{\text{Obiekt}_2}, \dots, \underbrace{(\text{Poz}_N, \text{Prę}_N)}_{\text{Obiekt}_N} \right] \quad (1.1)$$

Dlatego też aplikacje zorientowane na wysoką wydajność obliczeniową korzystają z ułożenia SoA, czyli struktury tablic (ang. *Structure of Arrays*), w której atrybuty tego samego typu tworzą oddzielne, ciągłe obszary pamięci.[4]

$$\text{SoA} = \begin{cases} \text{Tablica pozycji} = [\text{Poz}_1, \text{Poz}_2, \dots, \text{Poz}_N] \\ \text{Tablica prędkości} = [\text{Prę}_1, \text{Prę}_2, \dots, \text{Prę}_N] \end{cases} \quad (1.2)$$

Jeżeli algorytm odwołuje się do pamięci w sposób losowy lub nieliniowy, czyli tak jak ma to miejsce w przypadku układu tablic struktur, wczytana 64-bajtowa linia będzie zawierać wiele bezużytecznych w danym momencie informacji. To sprawia, że procesor szybko wykorzysta całą pojemność pamięci podręcznej pierwszego poziomu.

Z drugiej strony, gdy dane są ułożone w pamięci sekwencyjnie (układ struktur tablic), pobranie pierwszego elementu spowoduje załadowanie kolejnych elementów do pamięci L1. Dzięki temu rozwiązaniu procesor, wykonując instrukcje wektorowe, znajdzie kolejne potrzebne dane już w najszybszej pamięci.[4]

Aby zilustrować, w jaki sposób procesor i jego pamięć podręczna widzą układ tych danych podczas operacji wymagającej odczytu wyłącznie jednego atrybutu (np. samej

pozycji, z całkowitym pominięciem prędkości), można zestawiać ze sobą oba podejścia. W przypadku tradycyjnego modelu AoS, wskaźnik przechodzący przez pamięć musi załadować pełen obiekt, odczytać pozycję, a następnie przeskoczyć niepotrzebną w danej chwili prędkość. Generuje to narzut i zanieczyszcza linię pamięci podręcznej:

$$\text{Odczyt AoS: } \left[\underbrace{\mathbf{Poz}_1}_{\text{odczyt}}, \underbrace{\text{Prę}_1}_{\text{omijane}}, \underbrace{\mathbf{Poz}_2}_{\text{odczyt}}, \underbrace{\text{Prę}_2}_{\text{omijane}}, \dots, \underbrace{\mathbf{Poz}_N}_{\text{odczyt}}, \underbrace{\text{Prę}_N}_{\text{omijane}} \right] \quad (1.3)$$

Z kolei w architekturze SoA odczytywane są wyłącznie pożądane informacje. Procesor sprawnie pobiera do rejestrów spójny, ciągły wektor jednorodnych danych, podczas gdy niezwiązane z daną operacją tablice pozostają całkowicie nietknięte i nie obciążają przepustowości pamięci:

$$\text{Odczyt SoA: } \left\{ \begin{array}{l} \left[\mathbf{Poz}_1, \mathbf{Poz}_2, \dots, \mathbf{Poz}_N \right] \\ \text{ciągły odczyt (tylko potrzebne dane)} \\ \left[\text{Prę}_1, \text{Prę}_2, \dots, \text{Prę}_N \right] \\ \text{całkowicie nietknięte bloki pamięci} \end{array} \right. \quad (1.4)$$

1.2 GPS

Global Positioning System (GPS), oficjalnie znany jako NAVSTAR GPS, to satelitalny system nawigacyjny stworzony i zarządzany przez Departament Obrony Stanów Zjednoczonych. Należy on do szerszej rodziny systemów klasy GNSS (ang. *Global Navigation Satellite System*), do której zaliczają się również europejski system Galileo, rosyjski GLONASS oraz chiński BeiDou. Choć w mowie potocznej akronim „GPS” stosuje się uniwersalnie do określenia każdego rodzaju nawigacji satelitarnej, w dosłownym ujęciu odnosi się on do konkretnego amerykańskiego systemu satelitarnego.[5]

1.2.1 Zasada działania

Zasada działania opiera się na pomiarze czasu propagacji sygnału. Jeżeli odbiornik wie, o której godzinie sygnał został wysłany przez satelitę i o której dotarł do anteny, może obliczyć czas przelotu (ang. *Time of Flight*). Odległość od satelity to iloczyn czasu przelotu i prędkości światła. Odległość ta determinuje miejsce w przestrzeni, gdzie znajduje się odbiornik.[5]

W ujęciu matematycznym, jeśli satelita emituje sygnał w czasie t^s , a odbiornik rejestruje go w czasie t^r , to teoretyczna odległość geometryczna d jest wyrażana wzorem:

$$d = c \cdot (t^r - t^s) \quad (1.5)$$

gdzie c oznacza prędkość światła. W praktyce jednak metoda ta wymaga uwzględnienia dodatkowych czynników. Wyznaczenie odległości od jednego satelity lokalizuje odbiornik na powierzchni sfery. Zastosowanie dwóch satelitów zawęży pozycję do okręgu stanowiącego przecięcie dwóch sfer, a z trzech – do dwóch punktów w przestrzeni.

Aby jednoznacznie określić swoją pozycję w trójwymiarowym układzie współrzędnych (np. WGS 84), odbiornik musi zatem wyznaczyć cztery niewiadome: współrzędne przestrzenne x, y, z oraz błąd zegara δt . Wymaga to jednoczesnego odbioru sygnału z minimum czterech satelitów, co pozwala na ułożenie i rozwiązanie układu równań składającego się z czterech równań nieliniowych w postaci:[5]

$$\rho = \sqrt{(x_1 - x)^2 + (y_1 - y)^2 + (z_1 - z)^2} + c \cdot \delta t \quad (1.6)$$

1.2.2 Formaty danych GPS

Wraz z popularyzacją systemów informacji geograficznej (GIS) i osobistych urządzeń nawigacyjnych, zaistniała potrzeba ustandaryzowania zapisu w sposób umożliwiający łatwą wymianę informacji między różnymi platformami. Doprowadziło to do powstania formatów opartych na języku znaczników XML (ang. *Extensible Markup Language*), z których najpowszechniejszym stał się GPX (ang. *GPS Exchange Format*).

GPX zdefiniował podstawowy system opisu tras (ang. *Routes*), śladów (ang. *Tracks*) i punktów (*Waypoints*). Jednakże, wraz z rozwojem telemetrii sportowej i smartwatchy, nawigacyjny charakter formatu GPX okazał się niewystarczający, co wymusiło stworzenie nowych formatów, integrujących dane przestrzenne z fizjologicznymi.[6]

Ewolucja ta poszła w dwóch głównych kierunkach. Z jednej strony, na potrzeby urządzeń sportowych opracowano wydajny, binarny format FIT (ang. *Flexible and Interoperable Data Transfer*). Dzięki swojej skompresowanej strukturze pozwala on na bardzo oszczędny zapis danych lokalizacyjnych oraz parametrów biomechanicznych i fizjologicznych (takich jak moc czy tętno) bezpośrednio w pamięci zegarków i liczników rowerowych.[7] Z drugiej strony, naturalnym krokiem było rozwinięcie dotychczasowej technologii XML w stronę sportu. Tak powstał format TCX (ang. *Training Center XML*), który stał się bezpośrednim łącznikiem między czysto geograficznym GPX-em a nowoczesną telemetrią treningową.

Format TCX (Training Center XML) został zaprojektowany i wprowadzony przez firmę Garmin w 2007 roku jako część oprogramowania Garmin Training Center. W przeciwieństwie do uniwersalnego formatu GPX, TCX został zbudowany od podstaw z myślą o analizie sportowej, rekreacji i monitorowaniu wysiłku fizycznego.

Z punktu widzenia inżynierii danych, plik TCX jest dokumentem XML podlegającym walidacji przez schemat XSD (XML Schema Definition), opublikowany w przestrzeni nazw <http://www.garmin.com/xmlschemas/TrainingCenterDatabase/v2>. [8]

Listing 1.1: Przykładowy plik TCX

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <TrainingCenterDatabase
3   xsi:schemaLocation="http://www.garmin.com/xmlschemas/
   TrainingCenterDatabase/v2 http://www.garmin.com/xmlschemas/
   TrainingCenterDatabasev2.xsd"
4   xmlns:ns5="http://www.garmin.com/xmlschemas/ActivityGoals/v1"
5   xmlns:ns3="http://www.garmin.com/xmlschemas/ActivityExtension/
   v2"
6   xmlns:ns2="http://www.garmin.com/xmlschemas/UserProfile/v2"
7   xmlns="http://www.garmin.com/xmlschemas/TrainingCenterDatabase
   /v2"
8   xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
   xmlns:ns4="http://www.garmin.com/xmlschemas/
   ProfileExtension/v1">
9 <Activities>
10 <Activity Sport="Running">
11 <Id>2025-05-06T04:58:19.000Z</Id>
12 <Lap StartTime="2025-05-06T04:58:19.000Z">
13 <TotalTimeSeconds>335.479</TotalTimeSeconds>
14 <DistanceMeters>1000.0</DistanceMeters>
15 <MaximumSpeed>3.4149999618530273</MaximumSpeed>
16 <Calories>58</Calories>
17 <AverageHeartRateBpm>
18 <Value>146</Value>
19 </AverageHeartRateBpm>
20 <MaximumHeartRateBpm>
21 <Value>158</Value>
22 </MaximumHeartRateBpm>
23 <Intensity>Active</Intensity>
24 <TriggerMethod>Manual</TriggerMethod>
25 <Track>
26 <Trackpoint>
27 <Time>2025-05-06T04:58:19.000Z</Time>
28 <Position>
29 <LatitudeDegrees>51.23510423116386</
   LatitudeDegrees>
30 <LongitudeDegrees>22.55628313869238</

```

```

31         LongitudeDegrees>
32     </Position>
33     <AltitudeMeters>171.39999389648438</AltitudeMeters>
34     <DistanceMeters>0.8999999761581421</DistanceMeters>
35     <HeartRateBpm>
36         <Value>113</Value>
37     </HeartRateBpm>
38     <Extensions>
39         <ns3:TPX>
40             <ns3:Speed>0.8679999709129333</ns3:Speed>
41             <ns3:RunCadence>0</ns3:RunCadence>
42         </ns3:TPX>
43     </Extensions>
44     </Trackpoint>
45     <Trackpoint>
46         </Track>
47     </Lap>
48 </Activity>
49 </Activities>
</TrainingCenterDatabase>

```

Główną cechą odróżniającą TCX od innych formatów jest jego orientacja na „aktywność” (ang. *Activity*), a nie tylko i wyłącznie na „ścieżkę” (ang. *Track*). Plik TCX modeluje zbiór danych telemetrycznych w oparciu o zagnieżdżoną hierarchię:

- TrainingCenterDatabase: Główny węzeł dokumentu, który zawiera w sobie aktywności użytkownika.
- Activities / Activity: Reprezentuje pojedynczy trening. Atrybut Sport węzła Activity natychmiast kategoryzuje rodzaj ruchu (np. Running, Biking).
- Lap (Okrążenie): Jednostka podziału treningu w TCX. TCX dzieli aktywność na logiczne okrążenia (definiowane ręcznie przez użytkownika za pomocą przycisku na zegarku lub automatycznie, np. co 1 kilometr). Węzeł Lap przechowuje od razu zagregowane statystyki: TotalTimeSeconds (całkowity czas), DistanceMeters (dystans), MaximumSpeed (prędkość maksymalna) oraz Calories (spalone kalorie) itp.
- Track / Trackpoint: Wewnątrz każdego okrążenia znajduje się ścieżka, stanowiąca zbiór punktów pomiarowych (Trackpoint).[8]

Pojedynczy punkt pomiarowy w formacie TCX stanowi połączenie danych z różnych sensorów. Podczas gdy tradycyjny odbiornik GPS dostarcza głównie współrzędnych i

czasu, zegarek sportowy synchronizuje w tym samym momencie dane z pulsometru, barometru i akcelerometru.

Struktura informacyjna węzła Trackpoint obejmuje:

- **<Time>**: Znacznik czasu w formacie ISO 8601 (np. 2023-05-14T08:30:00.000Z).
- **<Position>**: Blok przestrzenny zawierający **<LatitudeDegrees>** i **<LongitudeDegrees>**. Reprezentacja bazuje na układzie WGS 84 w formie stopni dziesiętnych. Element ten jest opcjonalny – np. w przypadku biegu na bieżni mechanicznej, zegarek generuje punkty TCX pozbawione bloku **<Position>**, ale zawierające tętno i czas.
- **<AltitudeMeters>**: Wysokość, najczęściej z dokładnego czujnika barometrycznego kalibrowanego sygnałem GPS.
- **<DistanceMeters>**: Skumulowany dystans od początku okrążenia.
- **<HeartRateBpm>**: Informacja o tętnie w uderzeniach na minutę.[8]

Z uwagi na fakt, że oryginalny schemat TCX powstał kilkanaście lat temu, nie przewidywał on nowoczesnych metryk, takich jak moc generowana podczas biegu, balans stóp czy saturacja krwi. Aby uniknąć konieczności ciągłego modyfikowania głównego schematu, Garmin wprowadził mechanizm rozszerzeń w oparciu o nowe przestrzenie nazw (**ActivityExtension/v2**). Wewnątrz węzła Trackpoint można umieścić blok **<Extensions>**, który pozwala na dołączanie dowolnych nowych danych.[8]

Rozdział 2

HPC i Python

Ten rozdział poruszy tematy dotyczące architektury systemów High-Performance Computing (HPC) i specyfiki języka Python w celu zrozumienia jego ograniczeń wydajnościowych. Przybliżone zostaną również metody integracji standardowej implementacji z wydajnymi bibliotekami opartymi na niskopoziomowych rozwiązaniach.

2.1 HPC

Przetwarzanie o wysokiej wydajności (ang. *High-Performance Computing*) to dziedzina informatyki zajmująca się agregacją mocy obliczeniowej w celu rozwiązywania złożonych problemów naukowych i inżynierskich, których realizacja na zwykłych komputerach byłaby niemożliwa ze względu na ograniczenia czasowe lub pamięciowe.

Systemy HPC, potocznie nazywane superkomputerami, pozwalają na wykonywanie bilionów, a nawet trylionów obliczeń w ciągu sekundy. Podstawową miarą wydajności w systemach HPC jest wskaźnik FLOPS (ang. *Floating-Point Operations Per Second*), określający liczbę operacji zmiennoprzecinkowych wykonywanych w ciągu jednej sekundy. Współczesne superkomputery osiągają wydajność rzędu petaFLOPS (10^{15} operacji na sekundę), a najpotężniejsze jednostki na świecie przekroczyły już barierę exaFLOPS (10^{18} operacji na sekundę).[9]

Współczesne systemy HPC rzadko są pojedynczymi, monolitycznymi maszynami. Opierają się na architekturze klastrowej, w której setki lub tysiące niezależnych serwerów jest połączonych w jedną, zintegrowaną strukturę. Podstawowe elementy sprzętowe klastra HPC obejmują:

- Węzły obliczeniowe (ang. *Compute Nodes*): Odpowiedzialne za wykonywanie właściwych obliczeń. Każdy węzeł to w praktyce autonomiczny serwer wyposażony we własne procesory wielordzeniowe, pamięć operacyjną oraz coraz częściej w akcelerator obliczeniowe, takie jak procesory graficzne.[10]

- Węzły dostępne (ang. *Login Nodes*): Są to punkty wejściowe do systemu. Użytkownik loguje się do węzła dostępowego, aby kompilować kod, zarządzać swoimi danymi oraz zlecać zadania obliczeniowe. Węzły te nie służą do wykonywania ciężkich obliczeń, aby nie blokować dostępu innym użytkownikom.[10]
- Sieć połączeń: Ponieważ zadania w HPC są dzielone na wiele węzłów, które muszą nieustannie wymieniać między sobą dane, standardowe sieci Ethernet mogą okazać się niewystarczające. W systemach HPC stosuje się dedykowane sieci charakteryzujące się bardzo wysoką przepustowością i niskimi opóźnieniami. Dominującą technologią w tym obszarze jest InfiniBand.[3]
- Systemy pamięci masowej: Obliczenia HPC generują i konsumują terabajty danych. Zwykle dyski twarde nie są w stanie poradzić sobie z takim wyzwaniem. Stosuje się więc systemy plików (np. Lustre), które pozwalają tysiącom procesów na jednoczesny odczyt i zapis danych, rozpraszając fizyczny zapis na setki dysków i serwerów plikowych.[10]

Aby w pełni wykorzystać potencjał klastra HPC, oprogramowanie musi zostać zaprojektowane tak, by działało równolegle. Wyróżnia się trzy główne paradygmaty programowania w takich środowiskach:

- Pamięć współdzielona (ang. *Shared Memory*): Wykorzystywana w obrębie pojedynczego węzła. Wiele wątków tego samego programu wykonuje się na różnych rdzeniach procesora, ale wszystkie mają dostęp do tej samej fizycznej przestrzeni pamięci. Standardem programistycznym dla tego modelu jest OpenMP.[3]
- Pamięć rozproszona (ang. *Distributed Memory*): Konieczna, gdy program wykonuje się na wielu różnych węzłach, z których każdy posiada własną, niezależną pamięć. Węzły muszą „rozmawiać” ze sobą, przesyłając komunikaty przez sieć połączeń. De facto standardem w tej dziedzinie jest biblioteka MPI (ang. *Message Passing Interface*).[3]
- Przetwarzanie heterogeniczne (ang. *Heterogeneous Computing*): Gdy obliczenia zlecano są do kart graficznych, stosuje się specyficzne języki i biblioteki, takie jak CUDA (dla środowiska NVIDIA) lub otwarty standard OpenCL.[3]

W środowisku HPC kod źródłowy to jedynie połowa sukcesu. Równie istotny jest sam proces tłumaczenia go na język maszynowy. Ośrodki superkomputerowe rzadko opierają się wyłącznie na standardowych kompilatorach typu open-source, często inwestując w kompilatory dedykowane konkretnym architekturze sprzętowej.

Kompilatory stworzone dla takich firm jak Intel, NVIDIA, czy AMD są w stanie wygenerować zestaw instrukcji dopasowany do mikroarchitektury procesora, na którym kod będzie uruchamiany.

2.1.1 Języki stosowane w HPC

W systemach HPC priorytetem jest absolutna maksymalizacja wydajności oraz minimalizacja czasu wykonania i zużycia pamięci. Z tego powodu dominującą rolę odgrywają języki kompilowane (tzw. języki niskiego poziomu), które oferują programiście bezpośrednią kontrolę nad zarządzaniem pamięcią operacyjną oraz architekturą sprzętową, a jednocześnie pozwalają kompilatorom na przeprowadzanie znacznej optymalizacji kodu wynikowego.

Język C jest powszechnie stosowany ze względu na wysoką wydajność, zwężłość składni i minimalny narzut abstrakcji (ang. *overhead*). Operowanie na poziomie wskaźników pozwala na ścisłą integrację kodu z architekturą sprzętową. Ułatwia to optymalizację algorytmów oraz pozwala kompilatorom na skuteczną wektoryzację pętli. Ponadto język C pełni rolę uniwersalnego standardu w tworzeniu interfejsów binarnych, co umożliwia bezproblemową integrację oprogramowania i bibliotek tworzonych w innych technologiach.

Z kolei język C++ rozwija te możliwości, łącząc bazową wydajność języka C z paradygmatami programowania obiektowego. W kontekście HPC istotne znaczenie ma zastosowanie szablonów, które pozwalają na przeniesienie części logiki decyzyjnej z czasu wykonania programu na czas jego kompilacji. Umożliwia to projektowanie elastycznych struktur danych bez dodatkowego obciążenia wydajnościowego. Współczesne standardy języka C++ dostarczają również natywne mechanizmy współbieżności, wspierając tym samym rozwój zaawansowanych aplikacji wielowątkowych.[11]

2.1.2 Wbudowane struktury danych języka Python a wydajność obliczeniowa

Język Python, dzięki swojej elastyczności, dynamicznemu typowaniu oraz niezwykle czytelnej składni, stał się jednym z najpopularniejszych języków programowania na świecie. Jednak mechanizmy, które czynią go tak przystępnym w procesie tworzenia oprogramowania, stanowią jednocześnie jego największą barierę w kontekście przetwarzania o wysokiej wydajności. Standardowe implementacje opierają się na architekturze, która przedkłada uniwersalność nad optymalizację sprzętową.

W językach niskiego poziomu, takich jak C czy C++, standardowa tablica jest ciągłym blokiem pamięci przechowującym wartości jednego, z góry określonego typu (np. 64-bitowe liczby całkowite), co zapewnia lokalność przestrzenną oraz odgórnie znany rozmiar. Wbudowana lista w języku Python działa na zupełnie innej zasadzie. Pod względem implementacyjnym nie jest to tablica przechowująca bezpośrednio dane, lecz dynamiczna tablica wskaźników. Oznacza to, że każdy element na liście jest jedynie adresem lub inaczej odniesieniem do rzeczywistego obiektu umieszczonego w pamięci operacyjnej komputera.[12]

Dodatkowo, z uwagi na dynamiczne typowanie i fakt, że w Pythonie wszystko jest obiektem, nawet prosta liczba całkowita nie jest tylko ciągiem bitów. W standardowej implementacji Pythona, każda liczba jest strukturą `PyObject`, która oprócz samej wartości zawiera minimum dwa dodatkowe pola:

- Licznik referencji (ang. *reference count*) – przechowujący informację o liczbie zmiennych wskazujących na dany obiekt.
- Wskaźnik na typ obiektu – pozwalający interpreterowi określić w czasie wykonywania, z jakim typem danych ma do czynienia.[12]

Taka architektura generuje spory narzut pamięciowy (ang. *memory overhead*). Podczas gdy pojedyncza, 64-bitowa liczba całkowita w języku C zajmuje dokładnie 8 bajtów, ten sam obiekt w Pythonie może zajmować znacznie więcej bajtów w zależności od potrzeby, a do tego dochodzą jeszcze 8-bajtowe wskaźniki w samej liście. W przypadku przetwarzania milionów punktów z danych GPS, ilość alokowanej pamięci drastycznie rośnie, obciążając przepustowość szyny pamięci.

Jeszcze poważniejszą konsekwencją jest wpływ list na pamięć podręczną procesora. Jak wspomniano w rozdziale poświęconym hierarchii pamięci, procesory osiągają maksymalną wydajność, gdy dane są ułożone sekwencyjnie. Przetwarzanie natywnej listy w Pythonie zmusza procesor do wykonywania skoków. Odczyt elementu wymaga najpierw pobrania wskaźnika, a następnie wyciągnięcia informacji o nim, co prowadzi do odczytu danych z losowego, nieprzewidywalnego adresu w pamięci RAM.[4]

Prowadzi to do nieustannych "chybień" pamięci podręcznej (ang. *cache misses*). Linie pamięci cache są wypełniane bezużytecznymi danymi, a procesor spędza większość cykli zegara na bezczynnym oczekiwaniu na dostarczenie danych z wolnej pamięci głównej RAM. Architektura ta jest klasycznym przykładem skrajnie rozproszonego układu, przeciwnego do optymalnego wzorca Structure of Arrays (SoA).[2]

Z perspektywy wektoryzowania lub zrównoleglenia na poziomie danych, natywne listy Pythona są praktycznie nieużyteczne dla kompilatorów i procesorów wektorowych. Mechanizmy wektoryzacji wymagają, aby instrukcje były aplikowane do paczek jednorodnych, przylegających do siebie w pamięci danych [3].

Ponieważ lista w Pythonie może przechowywać elementy różnych typów w sposób dynamiczny (np. liczby całkowite, zmiennoprzecinkowe i ciągi znaków obok siebie), kompilator JIT lub sprzęt nie są w stanie z góry zagwarantować spójności typów w czasie wykonania. Przed każdą operacją arytmetyczną interpreter musi przeprowadzić kosztowną procedurę dynamicznego sprawdzania typów (ang. *type checking*) dla każdego pojedynczego obiektu. To skutecznie blokuje możliwość wykorzystania szerokich rejestrów wektorowych współczesnych architektur HPC.[13]

W celu ominięcia tych barier architektonicznych, biblioteki analityczne zorientowane na wydajność zmuszone są do omijania wewnętrznych struktur Pythona i implementowania własnych, ciągłych bloków pamięci w niższych warstwach (np. poprzez język C), co stanowi fundament działania takich narzędzi jak NumPy.

2.1.3 NumPy

NumPy to jedna z najważniejszych i najbardziej fundamentalnych bibliotek języka Python, przeznaczona do zaawansowanych obliczeń matematycznych. Stanowi ona podstawę dla wielu innych popularnych narzędzi do analizy danych i uczenia maszynowego, takich jak Pandas, SciPy, Scikit-Learn czy TensorFlow. Głównym celem powstania pakietu NumPy było dostarczenie wydajnych struktur danych oraz zoptymalizowanych algorytmów do operacji na dużych zbiorach informacji, co w zwykłym języku Python jest procesem mało wydajnym ze względu na jego interpretowany charakter i dynamiczne typowanie.[14]

Tablice wielowymiarowe i podejście do typów danych

Sercem biblioteki jest obiekt `ndarray` (ang. *N-dimensional array*), który reprezentuje wielowymiarową tablicę elementów. W przeciwieństwie do standardowego języka Python, gdzie listy mają charakter dynamiczny i mogą przechowywać obiekty różnych typów jednocześnie, biblioteka NumPy wprowadza rygorystyczne podejście do zarządzania danymi. Tablica `ndarray` jest z definicji homogeniczna – wszystkie jej elementy muszą reprezentować ten sam typ, który jest określany przez atrybut `dtype`. [14]

NumPy oferuje rozbudowany zestaw zdefiniowanych typów numerycznych, opartych na standardach znanych z języka C (np. `int32`, `int64`, `float64`). Zdefiniowanie konkretnego typu pozwala na alokację całej tablicy w ciągłym bloku pamięci operacyjnej. Eliminuje to konieczność przechowywania metadanych dla każdego pojedynczego elementu, co ma miejsce w wbudowanych listach Pythona. Ta jednorodność i ciągłość pamięciowa skutkują znacznym zmniejszeniem narzutu pamięciowego oraz pozwalają na błyskawiczny dostęp do danych z poziomu pamięci podręcznej procesora. [14]

Wektoryzacja w obliczeniach numerycznych

Ważnym paradygmatem determinującym wysoką wydajność biblioteki NumPy jest wektoryzacja (ang. *vectorization*). Tradycyjne operacje na kolekcjach danych w zwykłym Pythonie wymagają stosowania jawnych pętli iteracyjnych (np. pętli `for`), co w przypadku wielowymiarowych zbiorów danych prowadzi do ogromnego spadku wydajności.

Wektoryzacja polega na zastąpieniu tych pętli operacjami aplikowanymi na całe kolekcje danych. Znaczna część biblioteki NumPy została zaimplementowana w kompilowa-

nych, niskopoziomowych językach C oraz Fortran. Dzięki temu operacje matematyczne omijają narzut związany z interpretacją kodu w maszynie wirtualnej Pythona. Co więcej, algorytmy NumPy są zaprojektowane tak, aby wykorzystywać sprzętowe instrukcje wektorowe nowoczesnych procesorów. Pozwala to na wektoryzowanie i zrównoleglenie obliczeń na poziomie sprzętowym, np. poprzez jednoczesne dodawanie wielu elementów dwóch wektorów w jednym takcie procesora.[14]

Broadcasting

Niezwykle istotną cechą biblioteki NumPy jest wbudowany mechanizm transmitowania (ang. *broadcasting*). Umożliwia on wykonywanie operacji arytmetycznych na tablicach o różnych, ale kompatybilnych wymiarach.

W standardowej algebrze liniowej operacje na macierzach wymagają ścisłego dopasowania ich kształtów. NumPy potrafi zinterpretować intencję programisty i w przypadku braku dopasowania "rozszerza" mniejszą tablicę, aby jej wymiary odpowiadały większej. Dzieje się to bez fizycznego kopiowania danych w pamięci, co pozwala na oszczędność zasobów systemowych podczas złożonych operacji na tablicach.[14]

Podstawowe operacje matematyczne i statystyczne

Pakiet dostarcza szeroki wachlarz wbudowanych, zoptymalizowanych metod, do których należą między innymi:

- Algebra liniowa: operacje na wektorach i macierzach, obliczanie wyznaczników, odwracanie macierzy oraz rozwiązywanie układów równań liniowych.
- Funkcje uniwersalne (ufuncs): szybkie, zwektoryzowane funkcje matematyczne stosowane element po elemencie (np. funkcje trygonometryczne, logarytmiczne, wykładnicze).
- Statystyka: wydajne wyznaczanie miar tendencji centralnej i rozproszenia, takich jak średnia, mediana, odchylenie standardowe czy wariancja dla wielkich zbiorów danych.[14]

Porównanie operacji w Pythonie a NumPy

Aby lepiej zobrazować różnicę pomiędzy tradycyjnym podejściem a

Listing 2.1: Porównanie operacji w Pythonie a NumPy

```
1 import numpy as np
2
3 ROZMIAR = 10000000
```

```
4
5 # =====
6 # 1. Standardowy Python (jawna petla for)
7 # =====
8 lista_a = list(range(ROZMIAR))
9 lista_b = list(range(ROZMIAR))
10
11 wynik_python = []
12 for i in range(len(lista_a)):
13     wynik_python.append(lista_a[i] + lista_b[i])
14
15 # =====
16 # 2. Biblioteka NumPy (wektoryzacja)
17 # =====
18 tablica_a = np.arange(ROZMIAR)
19 tablica_b = np.arange(ROZMIAR)
20
21 wynik_numpy = tablica_a + tablica_b
```

Powyższy listing 2.1 przedstawia różnicę w implementacji dodawania dwóch dużych zbiorów danych za pomocą standardowych mechanizmów języka Python oraz biblioteki NumPy. W pierwszej części (linie 6-13) operacja jest wykonywana przy użyciu tradycyjnej pętli `for`, która iteruje po każdym elemencie i dodaje go do nowej listy. W drugiej części (linie 16-21) zastosowano tablice NumPy. Dzięki mechanizmowi wektoryzacji dodawanie całych tablic sprowadza się do użycia operatora `+` bez konieczności jawnego pisania pętli. Kod z wykorzystaniem NumPy jest nie tylko krótszy i bardziej czytelny, ale przede wszystkim wykonuje się znacznie szybciej.

2.2 Pomocnicze biblioteki

Oprócz biblioteki NumPy, która stanowi fundament aplikacji, w projekcie wykorzystano również inne biblioteki pomocnicze. Pełnią one role na początkowych oraz końcowych etapach przetwarzania informacji – podczas pozyskiwania surowych danych oraz ich końcowej prezentacji. Są to biblioteki `lxml` oraz `matplotlib`.

Biblioteka `lxml` to jedno z najpopularniejszych narzędzi w języku Python przeznaczone do przetwarzania, analizy i modyfikacji dokumentów w formatach XML (ang. *Extensible Markup Language*) oraz HTML (ang. *Hypertext Markup Language*). Biblioteka oferuje mechanizmy parsowania iteracyjnego, co pozwala na przetwarzanie bardzo dużych plików "w locie", bez konieczności ładowania całego drzewa dokumentu do pamięci komputera.[15]

Z kolei biblioteka `matplotlib` to popularna biblioteka w Pythonie służąca do tworzenia wizualizacji danych. Biblioteka `matplotlib` służy do przekształcania suchych danych liczbowych w czytelne grafiki. Za jej pomocą generowane są przejrzyste wykresy i diagramy.[16]

Rozdział 3

Implementacja kodu

Ten rozdział poruszy zagadnienia związane z praktyczną realizacją założeń projektowych. Omówiona zostanie ogólna struktura projektu oraz jego najważniejsze komponenty. Następnie szczegółowo przedstawione zostaną dwa podejścia do realizacji kodu: przy użyciu standardowych struktur Pythona oraz z wykorzystaniem biblioteki NumPy.

3.1 Struktura projektu

Struktura projektu została podzielona na trzy główne katalogi. W folderze `data` przechowywane są surowe pliki wejściowe w formacie TCX, które stanowią materiał do analizy. Katalog `helping_scripts` zawiera narzędzia pomocnicze wykorzystywane podczas rozwoju projektu, w tym skrypt do generowania danych TCX (`generate.py`) oraz skrypty testujące wydajność (`np_test.py`, `py_test.py`). Z kolei główna logika znajduje się w folderze `scripts`. Tam znajdują się właściwe implementacje funkcji, które bazują na standardowych strukturach Pythona oraz tablicach NumPy, a także skrypt do parsowania plików TCX (`tcx_python.py`, `tcx_numpy.py`, `tcx_parser.py`). W tym folderze można znaleźć również skrypt mierzący wydajność czasową obu rozwiązań (`benchmark.py`) i moduł generujący wykresy z wynikami (`plot.py`).

- `data`
- `helping_scripts`:
 - `generate.py`
 - `np_test.py`
 - `py_test.py`
- `scripts`:
 - `benchmark.py`

- plot.py
- tcx_numpy.py
- tcx_parser.py
- tcx_python.py

3.2 Implementacja kodu do parsowania plików TCX

Plik `tcx_parser.py` ma zaimplementowane dwie funkcje parsujące. Pierwsza z nich (listing 3.1) opiera się na wbudowanych strukturach języka Python, natomiast druga (listing 3.2) wykorzystuje mechanizmy biblioteki NumPy.

Listing 3.1: Funkcja parsująca (Python)

```

1 def parser_py( file ):
2     ns = {
3         'tcx': 'http://www.garmin.com/xmlschemas/
           TrainingCenterDatabase/v2',
4         'ns3': 'http://www.garmin.com/xmlschemas/
           ActivityExtension/v2'
5     }
6     latitudes, longitudes, elevations, heart_rates, times,
       cadences = [], [], [], [], [], []
7     tree = etree.parse( file )
8
9     for tp in tree.iterfind( './tcx:Trackpoint', namespaces=ns ):
10         pos = tp.find( 'tcx:Position', namespaces=ns )
11         if pos is not None:
12             lat_elem = pos.find( 'tcx:Position/tcx:
                LatitudeDegrees', namespaces=ns )
13             lon_elem = pos.find( 'tcx:Position/tcx:
                LongitudeDegrees', namespaces=ns )
14             if lat_elem is not None and lon_elem is not None:
15                 time_elem = tp.find( 'tcx:Time', namespaces=ns )
16                 if time_elem is not None and time_elem.text:
17                     time_str = time_elem.text.replace( 'Z', '
                        +00:00' )
18                     times.append( datetime.fromisoformat( time_str
                        ).timestamp() )
19                 else:
```

```

20         times.append(0.0)
21         latitudes.append(float(lat_elem.text))
22         longitudes.append(float(lon_elem.text))
23         ele_elem = tp.find('tcx:AltitudeMeters',
24                             namespaces=ns)
25         elevations.append(float(ele_elem.text) if
26                             ele_elem is not None else 0.0)
27         hr_elem = tp.find(' ./tcx:HeartRateBpm/tcx:Value
28                             ', namespaces=ns)
29         heart_rates.append(float(hr_elem.text) if
30                             hr_elem is not None else 0.0)
31         cad_elem = tp.find(' ./ns3:RunCadence',
32                             namespaces=ns)
33         cadences.append(float(cad_elem.text) if cad_elem
34                             is not None else 0.0)
35
36     return latitudes, longitudes, elevations, heart_rates, times
37         , cadences

```

Działanie funkcji rozpoczyna się od zdefiniowania słowników z przestrzeniami nazw (ang. *namespaces*), które są wymagane przez bibliotekę lxml do poprawnego nawigowania po drzewie plików Garmin TCX (listing 3.1, linie 2-5). Następnie deklarowanych jest sześć pustych list, które będą przechowywać informacje o zmiennych treningowych (listing 3.1, linia 6). W kolejnym kroku wykorzystana została metoda `iterfind` (listing 3.1, linia 9). Tworzy ona generator, który przetwarza węzły Trackpoint "w locie". Kod weryfikuje również, czy dany punkt posiada znacznik Position oraz współrzędne geograficzne (listing 3.1, linie 10-14). Jeśli ich brakuje, cały węzeł jest pomijany. Właściwe dane są dopisywane do list za pomocą metody `.append()` (listing 3.1, linie 18, 21, 24, 26, 28). W przypadku braku odczytu do listy dopisywana jest wartość domyślna 0.0 (listing 3.1, linia 28).

Listing 3.2: Funkcja parsująca (NumPy)

```

1 def parser_np(file):
2     ns = "{http://www.garmin.com/xmlschemas/
3         TrainingCenterDatabase/v2}"
4     ns3 = "{http://www.garmin.com/xmlschemas/ActivityExtension/
5         v2}"
6     tree = etree.parse(file)
7     trackpoints = tree.findall(f'./{{ns}}Trackpoint')
8     n = len(trackpoints)

```

```

8   latitudes = np.empty(n, dtype=np.float64)
9   longitudes = np.empty(n, dtype=np.float64)
10  elevations = np.empty(n, dtype=np.float64)
11  heart_rates = np.empty(n, dtype=np.float64)
12  times = np.empty(n, dtype=np.float64)
13  cadences = np.empty(n, dtype=np.float64)
14
15  i = 0
16  for tp in trackpoints:
17      pos = tp.find(f'{ns}Position')
18      if pos is not None:
19          lat_str = pos.findtext(f'{ns}LatitudeDegrees')
20          lon_str = pos.findtext(f'{ns}LongitudeDegrees')
21          if lat_str is not None and lon_str is not None:
22              time_str = tp.findtext(f'{ns}Time')
23              if time_str is not None:
24                  times[i] = datetime.fromisoformat(time_str.
25                                                         replace('Z', '+00:00')).timestamp()
26              else:
27                  times[i] = np.nan
28                  latitudes[i] = float(lat_str)
29                  longitudes[i] = float(lon_str)
30                  ele_str = tp.findtext(f'{ns}AltitudeMeters')
31                  elevations[i] = float(ele_str) if ele_str is not
32                      None else np.nan
33                  hr_str = tp.findtext(f'://{ns}HeartRateBpm/{ns}
34                      Value')
35                  heart_rates[i] = float(hr_str) if hr_str is not
36                      None else np.nan
37                  cad_str = tp.findtext(f'://{ns3}RunCadence')
38                  cadences[i] = float(cad_str) if cad_str is not
39                      None else np.nan
40              i += 1
41
42  return latitudes[:i], longitudes[:i], elevations[:i],
43         heart_rates[:i], times[:i], cadences[:i]

```

W przeciwieństwie do leniwej iteracji z poprzedniej wersji, tutaj funkcja `findall` od razu pobiera wszystkie węzły typu `Trackpoint` do pamięci (listing 3.2, linia 5). Pozwala

to na określenie ich dokładnej liczby i przypisanie jej do zmiennej `n` (listing 3.2, linia 6). Znając liczbę punktów, tworzone są tablice przy pomocy funkcji `np.empty()` (listing 3.2, linie 8-13). Zmienna pomocnicza i pełni rolę licznika zapisanych punktów. Zamiast domyślnych wartości 0.0 dla brakujących atrybutów wpisywana jest wartość `np.nan` (listing 3.2, linie 22, 29, 31). Ostatecznie zwracane tablice są przycinane za pomocą składni `[:i]` (listing 3.2, linia 37), ponieważ ostateczna liczba zebranych punktów może być mniejsza niż zainicjalizowany na początku rozmiar tablic `n`.

3.3 Implementacja kodu przetwarzającego dane treningowe

Pliki `tcx_python.py` oraz `tcx_numpy.py` zawierają główną logikę projektu. To tutaj znajdują się funkcje odpowiedzialne za główną logikę projektu oraz przetwarzanie danych TCX.

3.3.1 Funkcje odpowiedzialne za obliczanie dystansu oraz przewyższeń

Listing 3.3: Funkcje odpowiedzialne za obliczanie dystansu oraz przewyższeń (Python)

```

1 def distance(lat1 , lon1 , lat2 , lon2):
2     phi1 , phi2 = math.radians(lat1) , math.radians(lat2)
3     delta_phi = math.radians(lat2 - lat1)
4     delta_lambda = math.radians(lon2 - lon1)
5     a = math.sin(delta_phi / 2.0) ** 2 + math.cos(phi1) * math.
        cos(phi2) * math.sin(delta_lambda / 2.0) ** 2
6     c = 2 * math.atan2(math.sqrt(a) , math.sqrt(1 - a))
7     return R * c
8
9 def total_distance(latitudes , longitudes):
10     total_distance = 0.0
11     for i in range(1 , len(latitudes)):
12         total_distance += distance(latitudes[i - 1] , longitudes[
            i - 1] , latitudes[i] , longitudes[i])
13     return total_distance
14
15 def elevation_gain(elevations):
16     gain = 0.0
17     for e1 , e2 in zip(elevations[:-1] , elevations[1:]):

```

```

18         diff = e2 - e1
19         if diff > 0:
20             gain += diff
21     return gain

```

W klasycznej implementacji Pythona (listing 3.3) wyznaczanie summarycznego dystansu oraz przewyższeń bazuje na pętlach iterujących przez kolekcje danych. Funkcja `distance` wykorzystuje moduł `math` do wyliczenia odległości ze wzoru Haversine’a, następnie `total_distance` łączy kolejne pary punktów i dodając je do sumy całkowitej. Z kolei `elevation_gain` liczy różnicę wysokości między parą punktów, po czym sumuje tylko wartości dodatnie.

Listing 3.4: Funkcje odpowiedzialne za obliczanie dystansu oraz przewyższeń (NumPy)

```

1 def distance_np(lat1, lon1, lat2, lon2):
2     phi1, phi2 = np.radians(lat1), np.radians(lat2)
3     delta_phi = np.radians(lat2 - lat1)
4     delta_lambda = np.radians(lon2 - lon1)
5     a = np.sin(delta_phi / 2.0) ** 2 + np.cos(phi1) * np.cos(
6         phi2) * np.sin(delta_lambda / 2.0) ** 2
7     c = 2 * np.arctan2(np.sqrt(a), np.sqrt(1 - a))
8     return R * c
9
10 def total_distance_np(latitudes, longitudes):
11     distances = distance_np(latitudes[:-1], longitudes[:-1],
12                             latitudes[1:], longitudes[1:])
13     return np.sum(distances)
14
15 def elevation_gain_np(elevations):
16     diffs = np.diff(elevations)
17     return np.sum(diffs[diffs > 0])

```

W przypadku biblioteki NumPy (listing 3.4) logika została znacznie uproszczona. Operacje matematyczne wywoływane są na całych wektorach wejściowych bez potrzeby jawnej iteracji. Obliczenia przewyższeń (listing 3.4, linie 13-15) wykorzystują funkcję `np.diff()`, która automatycznie zwraca wektor różnic wysokości, a dzięki zastosowaniu maski logicznej `diffs > 0` funkcja błyskawicznie sumuje wyłącznie wartości dodatnie.

3.3.2 Analiza parametru tętna serca

Listing 3.5: Funkcje odpowiedzialne za analizę tętna (Python)

```
1 def avg_hr(heart_rates):
2     total_hr = 0
3     count = 0
4     for hr in heart_rates:
5         if hr > 0:
6             total_hr += hr
7             count += 1
8     return total_hr / count if count > 0 else 0
9
10 def hr_zones(heart_rates, hr_max=185):
11     z1_min, z2_min, z3_min, z4_min, z5_min = [hr_max * p for p
12         in (0.5, 0.6, 0.7, 0.8, 0.9)]
13     zones = [0, 0, 0, 0, 0]
14     for hr in heart_rates:
15         if hr >= z5_min: zones[4] += 1
16         elif hr >= z4_min: zones[3] += 1
17         elif hr >= z3_min: zones[2] += 1
18         elif hr >= z2_min: zones[1] += 1
19         elif hr >= z1_min: zones[0] += 1
20     return zones
21
22 def elevation_hr(elevations, heart_rates):
23     climb_hr_total, climb_count, descent_hr_total, descent_count
24     = 0, 0, 0, 0
25     for e1, e2, hr in zip(elevations[:-1], elevations[1:],
26         heart_rates[1:]):
27         ele_diff = e2 - e1
28         if hr > 0:
29             if ele_diff > 0:
30                 climb_hr_total += hr
31                 climb_count += 1
32             else:
33                 descent_hr_total += hr
34                 descent_count += 1
35     avg_climb = climb_hr_total / climb_count if climb_count > 0
36     else 0
37     avg_descent = descent_hr_total / descent_count if
38     descent_count > 0 else 0
```

```
34 | return avg_climb , avg_descent
```

Pierwsza z funkcji, `avg_hr` (listing 3.5, linie 1–8), odpowiada za wyliczenie średniej wartości tętna z całej aktywności. Do całkowitej sumy oraz licznika pomiarów dodawane są wyłącznie wartości większe od zera.

Druga funkcja, `hr_zones` (listing 3.5, linie 9–19), ma za zadanie przyporządkować poszczególne odczyty do jednej z pięciu standardowych stref wysiłkowych, bazując na wartości tętna maksymalnego. Konstrukcja w linii 11 wylicza dolne progi tętna dla stref od Z1 do Z5, które stanowią od 50% do 90% tętna maksymalnego, a następnie przypisuje je do pięciu osobnych zmiennych. Z kolei w liniach 13–18 pętla iteruje po wszystkich pomiarach, przydzielając je do odpowiednich kategorii.

Ostatnia funkcja, `elevation_hr` (listing 3.5, linie 20–34), analizuje obciążenie organizmu w kontekście profilu wysokościowego pokonywanej trasy. Jej głównym celem jest podział danych o tętnie na odcinki wznoszące się oraz opadające. W każdej iteracji pętli program uzyskuje bezpośredni dostęp do dwóch sąsiadujących pomiarów wysokości i powiązanego z nimi tętna. Funkcja wylicza różnicę wysokości między punktami, a następnie przypisuje wartość tętna do odpowiedniego licznika w zależności czy nastąpił przyrost wysokości bądź jej spadek. Na koniec zwracane są średnie wartości tętna dla obu rodzajów nachylenia terenu.

Listing 3.6: Funkcje odpowiedzialne za analizę tętna (NumPy)

```
1 def avg_hr_np(heart_rates):
2     valid_hr = heart_rates[heart_rates > 0]
3     return np.mean(valid_hr) if valid_hr.size > 0 else 0
4
5
6 def hr_zones_np(heart_rates , hr_max=185):
7     bins = [hr_max * 0.5, hr_max * 0.6, hr_max * 0.7, hr_max *
8             0.8, hr_max * 0.9, np.inf]
9     counts, _ = np.histogram(heart_rates , bins=bins)
10    return counts.tolist()
11
12 def elevation_hr_np(elevations , heart_rates):
13     ele_diffs = np.diff(elevations)
14     hr_segments = heart_rates[1:]
15     valid_hr = hr_segments > 0
16     is_climb = ele_diffs > 0
17     is_descent_or_flat = ele_diffs <= 0
18     hr_on_climbs = hr_segments[valid_hr & is_climb]
```



```

19     hr_on_descents = hr_segments[valid_hr & is_descent_or_flat]
20     avg_climb = np.mean(hr_on_climbs) if hr_on_climbs.size > 0
        else 0
21     avg_descent = np.mean(hr_on_descents) if hr_on_descents.size
        > 0 else 0
22     return avg_climb, avg_descent

```

W pierwszej funkcji, `avg_hr_np` (listing 3.6, linie 1–3), proces filtrowania danych i obliczania średniej uległ znacznemu uproszczeniu. Druga z funkcji, `hr_zones_np` (listing 3.6, linie 6–9), kompletnie usunęła kaskadę instrukcji warunkowych. Zamiast tego, w linii 7 zdefiniowano tablicę `bins`, określającą krawędzie przedziałów dla poszczególnych stref tętna. Górną granicę ostatniej strefy (Z5) wyznaczono tu jako `np.inf`, aby przechwycić najwyższe odczyty. W linii 8, gdzie użyto funkcji `np.histogram()`. Przeprowadza ona grupowanie wszystkich wartości z tablicy `heart_rates` do zdefiniowanych przedziałów i zwraca gotową tablicę liczników wystąpień dla każdej strefy. Wynik ten jest konwertowany na standardową listę za pomocą metody `tolist()`.

Ostatnia funkcja, `elevation_hr_np` (listing 3.6, linie 12–22), została znacząco zmieniona względem standardowej implementacji. Funkcja `np.diff` automatycznie wylicza różnice między każdym elementem tablicy wysokości oraz jego sąsiadem. Następnie zdefiniowano zmienne przechowujące maski logiczne o poprawności tętna (`valid_hr`) oraz kierunku nacylenia trasy (`is_climb`, `is_descent_or_flat`). W liniach 18 i 19 te maski są nakładane na siebie przy użyciu operatora `&`.

3.3.3 Analiza wydolności biomechanicznej i tlenowej

Funkcja `perf_eff` (listing 3.7) to narzędzie analityczne, które pozwala na oszacowanie wydolności biomechanicznej i tlenowej. Kod ten został podzielony na trzy główne etapy: estymację mocy chwilowej, obliczenie mocy znormalizowanej oraz ocenę zmęczenia (rozprężenia tlenowego).

Listing 3.7: Analiza wydolności biomechanicznej i tlenowej (Python)

```

1 def perf_eff(latitudes, longitudes, elevations, heart_rates,
    times, weight=75.0):
2     delta_times = []
3     distances = []
4     speeds = []
5     grades = []
6     powers = []
7
8     for i in range(1, len(latitudes)):

```

```

9         delta_time = times[i] - times[i - 1]
10        if delta_time <= 0:
11            delta_time = 1.0
12
13        dist = distance(latitudes[i - 1], longitudes[i - 1],
14                        latitudes[i], longitudes[i])
15        ele_diff = elevations[i] - elevations[i - 1]
16        speed = dist / delta_time
17        grade = (ele_diff / dist) if dist > 0 else 0
18        power = (weight * 9.81 * speed * grade) + (0.5 * weight
19              * speed)
20        power = max(0, power)
21
22        delta_times.append(delta_time)
23        distances.append(dist)
24        speeds.append(speed)
25        grades.append(grade)
26        powers.append(power)
27
28    power_4th = []
29    window_size = 30
30    for i in range(len(powers)):
31        start = max(0, i - window_size + 1)
32        window = powers[start:i + 1]
33        rolling_mean = sum(window) / len(window)
34        power_4th.append(rolling_mean ** 4)
35
36    normalized_power = (sum(power_4th) / len(power_4th)) ** 0.25
37    if power_4th else 0
38    half = len(powers) // 2
39
40    def calculate_ef(power, hr):
41        avg_p = sum(power) / len(power) if power else 0
42        avg_hr = sum(hr) / len(hr) if hr else 1
43        return avg_p / avg_hr

```

```

44     ef_1 = calculate_ef(powers[:half], hr_data[:half])
45     ef_2 = calculate_ef(powers[half:], hr_data[half:])
46     decoupling = ((ef_1 - ef_2) / ef_1 * 100) if ef_2 > 0 else 0
47
48     return {
49         "Znormalizowana moc (W)": round(normalized_power, 2),
50         "Współczynnik wydajności (pierwsza połowa treningu)":
51             round(ef_1, 3),
52         "Współczynnik wydajności (druga połowa treningu)": round
53             (ef_2, 3),
54         "Aerobic decoupling": round(decoupling, 2)
55     }

```

Pierwszy etap odbywa się w pętli głównej (od linii 15). Program iteruje po wszystkich zapisanych punktach trasy, obliczając dla każdego kroku czas trwania, pokonany dystans, zmianę wysokości, prędkość oraz nachylenie terenu. Linia 28 odpowiada za estymację generowanej mocy. Wykorzystano uproszczony model matematyczny składający się z dwóch członów. Pierwszy z nich (`weight * 9.81 * speed * grade`) odpowiada za pracę przeciwko grawitacji na podjazdach i podbiegach. Drugi człon (`0.5 * weight * speed`) to szacunkowy koszt pokonywania oporów ruchu. W linii 29 zastosowano funkcję `max(0, power)`, aby zabezpieczyć funkcję przed wyliczeniem ujemnej mocy podczas stromych zjazdów.

Drugi etap (linie 26–34) to wyliczenie mocy znormalizowanej, która odzwierciedla fizjologiczny koszt wysiłku. W liniach 28–32 zastosowano algorytm średniej kroczącej z 30-sekundowym oknem czasowym, aby wygładzić chwilowe skoki mocy. Podniesienie każdej uśrednionej wartości do czwartej potęgi sprawia, że krótkie, ale intensywne momenty wysiłku otrzymują znacznie większą wagę w ostatecznym rozrachunku. Cały proces kończy się w linii 46 wyciągnięciem pierwiastka czwartego stopnia z sumy tych wartości, przywracając wynik do skali w watach.

Ostatni etap to analiza wskaźnika rozprężenia tlenowego (listing 3.7, linie 35–53). W tym celu została zdefiniowana funkcja pomocnicza `calculate_ef`, która wylicza współczynnik wydajności, który mówi o tym, ile watów mocy organizm generuje średnio na jedno uderzenie serca. Następnie dane z treningu dzielone są na dwie równe połowy. Obliczenie wydajności dla pierwszej i drugiej połowy pozwala na ich bezpośrednie zestawienie. W linii 46 wyliczany jest procentowy spadek tej wydajności w czasie. Jeśli tętno w drugiej połowie rośnie przy utrzymaniu tej samej mocy (lub moc spada przy tym samym tętnie), wskaźnik decoupling rośnie, co świadczy o narastającym zmęczeniu układu sercowo-naczyniowego. Całość zwracana jest w formie słownika z zaokrąglonymi wartościami.

Listing 3.8: Analiza wydolności biomechanicznej i tlenowej (NumPy)

```

1 def perf_eff_np(latitudes , longitudes , elevations , heart_rates ,
  times , weight=75.0):
2     delta_time = np.diff(times)
3     delta_time = np.maximum(delta_time , 1.0)
4
5     dist = distance_np(latitudes[:−1] , longitudes[:−1] ,
        latitudes[1:] , longitudes[1:])
6     ele_diff = np.diff(elevations)
7     speed = dist / delta_time
8
9     grade = np.divide(ele_diff , dist , out=np.zeros_like(ele_diff
        ), where=(dist > 0))
10
11     power = (weight * 9.81 * speed * grade) + (0.5 * weight *
        speed)
12     power = np.maximum(0 , power)
13
14     window_size = 30
15     window = np.ones(window_size) / window_size
16     rolling_mean = np.convolve(power , window , mode='valid')
17     normalized_power = np.mean(rolling_mean ** 4) ** 0.25 if len
        (rolling_mean) > 0 else 0.0
18
19     half = len(power) // 2
20     pow_1 , pow_2 = power[:half] , power[half:]
21
22     hr_data = heart_rates[1:]
23     hr_1 , hr_2 = hr_data[:half] , hr_data[half:]
24
25     avg_hr_1 = np.mean(hr_1) if hr_1.size > 0 else 0
26     avg_hr_2 = np.mean(hr_2) if hr_2.size > 0 else 0
27
28     ef_1 = np.mean(pow_1) / avg_hr_1 if avg_hr_1 > 0 else 0.0
29     ef_2 = np.mean(pow_2) / avg_hr_2 if avg_hr_2 > 0 else 0.0
30     decoupling = ((ef_1 − ef_2) / ef_1 * 100) if ef_1 > 0 else
        0.0
31

```

```
32     return {
33         "Znormalizowana - moc - (W)": round(float(normalized_power),
34         2),
35         "Współczynnik - wydajności - (pierwsza - połowa - treningu)":
36         round(float(ef_1), 3),
37         "Współczynnik - wydajności - (druga - połowa - treningu)": round
38         (float(ef_2), 3),
39         "Aerobic - decoupling": round(float(decoupling), 2)
40     }
```

W pierwszej kolejności wektoryzacji uległy podstawowe parametry fizyczne. Ręczne wyliczanie różnic czasu przeniesiono na funkcję `np.diff()`, a blokady przed wartościami ujemnymi zrealizowano za pomocą funkcji `np.maximum()`. Prędkość i dystans są teraz wyliczane jednorazowo dla całych tablic dzięki mechanizmowi krojenia list. Dodatkowo, klasyczne instrukcje warunkowe zapobiegające dzieleniu przez zero przy obliczaniu nachylenia terenu zastąpiono dzieleniem za pomocą funkcji `np.divide`. Wykorzystano w niej parametry `where`, który nakazuje dzielenie tylko dla dystansu większego od zera, oraz `out`, który automatycznie wstawia zera w pozostałych przypadkach.

Najbardziej znacząca zmiana zaszła w mechanizmie obliczania mocy znormalizowanej. Ręczne iterowanie po 30-sekundowym oknie czasowym całkowicie wyeliminowano na rzecz wbudowanej funkcji `np.convolve()`. Działa ona jak ruchome okno, które automatycznie przesuwają się po wszystkich odczytach mocy. Osiągamy w ten sposób identyczny efekt jak w przypadku tradycyjnej średniej kroczącej.

Znacznemu skróceniu uległa również końcowa analiza rozprężenia tlenowego. Usunięto wewnętrzną funkcję pomocniczą, a tablice mocy i tętna są teraz bezpośrednio dzielone na pół za pomocą indeksowania (wycinki `[:half]` oraz `[half:]`). Wartości wydajności wyliczane są od razu dla całych podzbiorów przez funkcję `np.mean()`.

Bibliografia

- [1] Ulrich Drepper, *What Every Programmer Should Know About Memory*, Red Hat, Inc., 2007
- [2] Agner Fog, *Optimizing software in C++: An optimization guide for Windows, Linux, and Mac platforms*, Agner Fog, 2025
- [3] John L. Hennessy, J. L., David A. Patterson, *Computer Architecture: A Quantitative Approach Fifth Edition*, Morgan Kaufmann, 2017
- [4] Richard Acton, *Data-Oriented Design: Software engineering for limited resources and short schedules*, Richard Acton, 2018
- [5] Bernhard Hofmann-Wellenhof, Herbert Lichtenegger, Elmar Wasle, *GNSS – Global Navigation Satellite Systems: GPS, GLONASS, Galileo, and more*, SpringerWien-NewYork, Austria, 2008
- [6] TopoGrafix, *GPX: the GPS Exchange Format*, <https://www.topografix.com/gpx.asp>, [dostęp: 14.04.2026]
- [7] Garmin, *Flexible and Interoperable Data Transfer (FIT) SDK*, <https://developer.garmin.com/fit/overview/>
- [8] Garmin, *Training Center Database XML Schema*, <https://www8.garmin.com/xmlschemas/TrainingCenterDatabasev2.xsd>, [dostęp: 14.04.2026]
- [9] TOP500, *TOP500's The List*, <https://www.top500.org/lists/top500/2025/11/>, [dostęp: 06.05.2026]
- [10] ETH Zurich, *HPC Documentation*, <https://docs.hpc.ethz.ch/>, [dostęp: 06.05.2026]
- [11] Björn Andrist, Viktor Sehr, *C++ High Performance: Master the art of optimizing the performance of your C++ applications*, Packt Publishing, Birmingham, 2018
- [12] Python Software Foundation, *Python 3 Documentation*, <https://docs.python.org/3/>, [dostęp: 06.05.2026]

- [13] Kevin Williams, Jason McCandless, David Gregg, *Dynamic Interpretation for Dynamic Scripting Languages*, Department of Computer Science, Trinity College Dublin, Dublin, 2009
- [14] NumPy Developers, *NumPy Documentation* <https://numpy.org/doc/2.4/>
- [15]
- [16] Matplotlib development team, *Matplotlib 3.10.9 documentation*, <https://matplotlib.org/stable/index.html>

Spis listingów

1.1	Przykładowy plik TCX	13
2.1	Porównanie operacji w Pythonie a NumPy	22
3.1	Funkcja parsująca (Python)	26
3.2	Funkcja parsująca (NumPy)	27
3.3	Funkcje odpowiedzialne za obliczanie dystansu oraz przewyższeń (Python)	29
3.4	Funkcje odpowiedzialne za obliczanie dystansu oraz przewyższeń (NumPy)	30
3.5	Funkcje odpowiedzialne za analizę tętna (Python)	30
3.6	Funkcje odpowiedzialne za analizę tętna (NumPy)	32
3.7	Analiza wydolności biomechanicznej i tlenowej (Python)	33
3.8	Analiza wydolności biomechanicznej i tlenowej (NumPy)	36